



[Home](#) / [Posts](#)

The 2026-07-28 MCP Specification Release Candidate

The release candidate for the next Model Context Protocol (MCP) specification is now available: a stateless protocol core, the Extensions framework, Tasks, MCP Apps, authorization hardening, and a formal deprecation policy.

May 21, 2026 · 9 min · David Soria Parra (Lead Maintainer), Den Delimarsky (Lead Maintainer)

► [Table of Contents](#)

The release candidate for MCP **2026-07-28** is now available. It is the largest revision of the protocol since launch and delivers on the [2026 roadmap](#):

- **a stateless core** that scales on ordinary HTTP infrastructure
- **extensions** including server-rendered UIs through [MCP Apps](#) and long-running work through the [Tasks extension](#)
- **authorization** that aligns more closely with OAuth and OpenID Connect deployments
- **a formal deprecation policy** so the protocol can evolve without breaking what you've built,

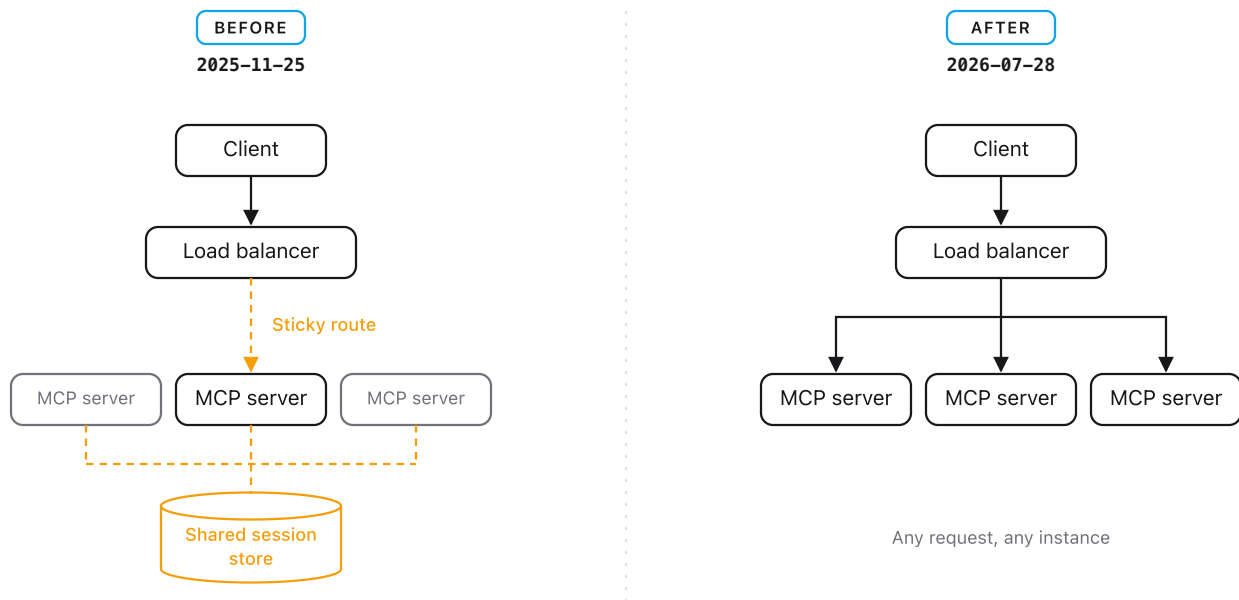
and many other changes.

The practical effect on a production deployment is immediate. A remote MCP server that previously needed sticky sessions, a shared session store, and deep packet inspection at the gateway can now run behind a plain round-robin load balancer, route traffic on an `Mcp-Method` header, and let clients cache `tools/list` responses for as long as the server's `ttlMs` permits.

The release candidate is available today and the final specification ships on **July 28, 2026**. This release contains breaking changes; see [Release Timeline and Validation](#) for the details.

A Stateless Protocol

The headline change is that MCP is now stateless at the protocol layer. Six [Specification Enhancement Proposals](#) (SEPs) work together to get there, completing the plan we laid out in [The Future of MCP Transports](#) in December.



Before and after

In [2025-11-25](#), calling a tool over Streamable HTTP means establishing a session first:

```
POST /mcp HTTP/1.1
Content-Type: application/json

{"jsonrpc": "2.0", "id": 1, "method": "initialize",
 "params": {"protocolVersion": "2025-11-25", "capabilities": {},
            "clientInfo": {"name": "my-app", "version": "1.0"}}
```

The server responds with an `Mcp-Session-Id` that every subsequent request must carry, pinning the client to whichever instance issued it:

```
POST /mcp HTTP/1.1
```

```
Mcp-Session-Id: 1868a90c-3a3f-4f5b
Content-Type: application/json
```

```
{"jsonrpc":"2.0","id":2,"method":"tools/call",
  "params":{"name":"search","arguments":{"q":"otters"}}}
```

In 2026-07-28 , the same call is a single self-contained request that any server instance can handle:

```
POST /mcp HTTP/1.1
MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call
Mcp-Name: search
Content-Type: application/json

{"jsonrpc":"2.0","id":1,"method":"tools/call",
  "params":{"name":"search","arguments":{"q":"otters"},
    "_meta":{"io.modelcontextprotocol/clientInfo":{"name":"my-app","version":
```

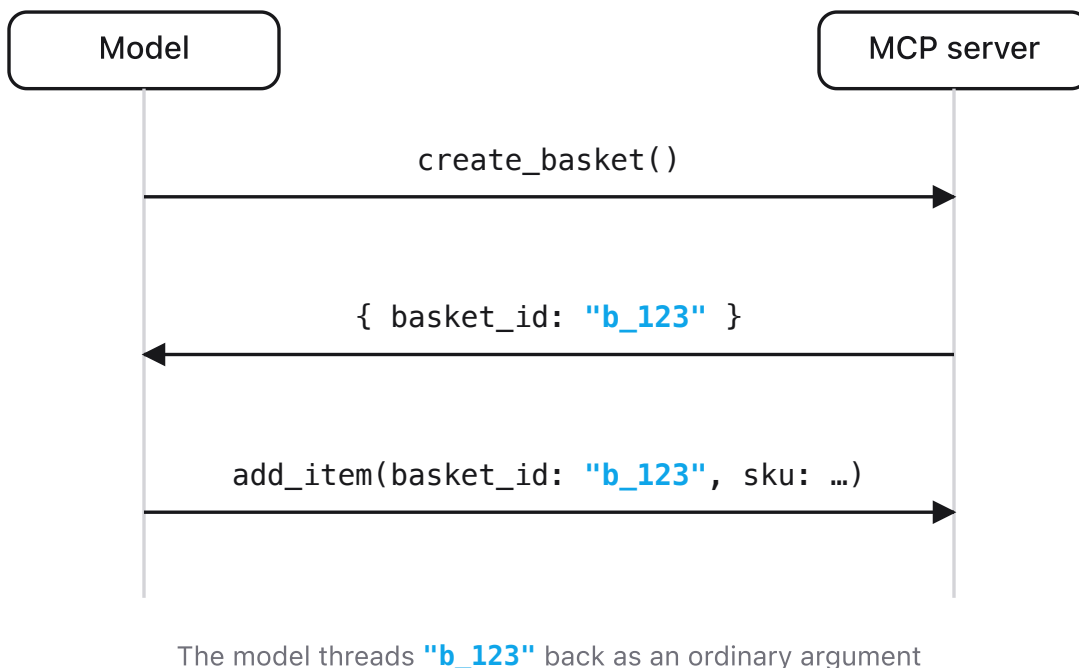
The handshake and session are gone

The `initialize / initialized` handshake is removed ([SEP-2575](#)). The protocol version, client info, and client capabilities that used to be exchanged once at connection time now travel in `_meta` on every request, and a new `server/discover` method lets clients fetch server capabilities when they need them up front.

The `Mcp-Session-Id` header and the protocol-level session that came with it are also removed ([SEP-2567](#)). With both gone, any MCP request can land on any server instance, and the sticky routing and shared session stores that horizontal deployments needed before are no longer required at the protocol layer.

Stateless protocol, stateful applications

Removing the protocol-level session does not mean your application has to be stateless. Servers that need to carry state across calls can do what HTTP APIs have always done: mint an explicit handle (a `basket_id` , a `browser_id`) from a tool and have the model pass it back as an ordinary argument on later calls.



In practice, we've found this pattern (the model threading an identifier from one tool call to the next) to be more than just a workable substitute for session state. It's often a more powerful one. The model can compose handles across tools, reason about them, and hand them off between steps in ways that externally managed session state, hidden in transport metadata, never really allowed.

The protocol no longer manages that state for you, but it doesn't prevent you from managing it yourself. The explicit-handle pattern simply makes the state visible to the model rather than hidden away.

Server-to-client requests, restructured

A stateless protocol still needs a way for servers to ask the client for something mid-call, such as an elicitation prompt. Two SEPs rebuild that flow so it works without a persistent connection.

Server-initiated requests may now only be issued while the server is actively processing a client request ([SEP-2260](#)). Earlier spec versions recommended this; it's now required. A user is never prompted out of nowhere, and every elicitation traces back to something they (or their agent) started.

Multi Round-Trip Requests ([SEP-2322](#)) change how those prompts are delivered. Instead of holding a Server-Sent Events (SSE) stream open, the server returns an

`InputRequiredResult` :

```
{
  "resultType": "input_required",
  "inputRequests": {
    "confirm": {
      "type": "elicitation",
      "message": "Delete 3 files?",
      "schema": { "type": "boolean" }
    }
  },
  "requestState": "eyJzdGVwIjoxLCJmaWxlcyl6WyJhIiwiaYiIsImMiXX0="
}
```

The client gathers the answers and re-issues the original call with `inputResponses` and the echoed `requestState` . Any server instance can pick that retry up because everything it needs is in the payload.

Routable, cacheable, traceable

Three smaller changes make the resulting traffic easier to operate.

The Streamable HTTP transport now requires `Mcp-Method` and `Mcp-Name` headers ([SEP-2243](#)) so load balancers, gateways, and rate-limiters can route on the operation without inspecting the body. Servers reject requests where the headers and body disagree.

List and resource read results now carry `ttlMs` and `cacheScope` ([SEP-2549](#)), modeled on HTTP `Cache-Control` . Clients know exactly how long a `tools/list` response is fresh and whether it's safe to share across users, and a long-lived SSE stream is no longer the only way to learn that a list changed.

W3C Trace Context propagation in `_meta` is now documented ([SEP-414](#)), locking down the `traceparent` , `tracestate` , and `baggage` key names so distributed traces correlate across SDKs and gateways. Several SDKs and tools were already doing this; with the key names fixed in the spec, a trace that starts in a host application can follow a tool call through the client SDK, the MCP server, and whatever the server calls downstream, and show up as a single span tree in an [OpenTelemetry](#)-compatible backend.

Extensions Become First-Class

Extensions existed in the [2025-11-25](#) release but had no formal process behind them.

[SEP-2133](#) adds that: extensions are identified by reverse-DNS IDs, negotiated through an `extensions` map on client and server capabilities, live in their own `ext-*` repositories with delegated maintainers, and version independently of the specification. A new Extensions Track in the SEP process gives them a path from experimental to official.

This release includes two official extensions.

MCP Apps: server-rendered user interfaces

[MCP Apps](#) ([SEP-1865](#)) lets servers ship interactive HTML interfaces that hosts render in a sandboxed iframe. Tools declare their UI templates ahead of time so hosts can prefetch, cache, and security-review them before anything runs. The rendered UI talks back to the host over the same JSON-RPC base protocol used everywhere else in MCP, so every UI-initiated action goes through the same audit and consent path as a direct tool call.

Tasks graduates to an extension

Tasks shipped as an experimental core feature in [2025-11-25](#). Production use surfaced enough redesign that the right home for it is an extension rather than the specification.

The [Tasks extension](#) reshapes the lifecycle around the stateless model: a server can answer `tools/call` with a task handle, and the client drives it with `tasks/get`, `tasks/update`, and `tasks/cancel`. Task creation is server-directed: the client advertises the extension and the server decides when a call should run as a task. `tasks/list` is removed because it can't be scoped safely without sessions.

Anyone who shipped against the [2025-11-25](#) experimental Tasks API will need to migrate to the new lifecycle.

Authorization Hardening

Six SEPs harden the [authorization specification](#) to align more closely with how OAuth 2.0 and OpenID Connect are deployed in practice.

Clients must now validate the `iss` parameter on authorization responses per [RFC 9207 \(SEP-2468\)](#). This is a low-cost mitigation for a class of mix-up attack that is more prevalent in MCP's single-client, many-server deployment pattern. In a future version, clients will be expected to reject responses that omit `iss`, so authorization servers should begin supplying it now if they don't already.

Clients now declare their OpenID Connect `application_type` during Dynamic Client Registration ([SEP-837](#)), avoiding the common case where an authorization server defaults a desktop or CLI client to `"web"` and rejects its localhost redirect URI. Clients bind registered credentials to the issuing authorization server's `issuer` and re-register when a resource migrates between authorization servers ([SEP-2352](#)). The spec also documents how to request refresh tokens from OpenID Connect-style authorization servers ([SEP-2207](#)), and clarifies scope accumulation during step-up ([SEP-2350](#)) and the `.well-known` discovery suffix ([SEP-2351](#)).

Roots, Sampling, and Logging Are Deprecated

Three core features are deprecated under the new [feature lifecycle policy](#) ([SEP-2577](#)):

Feature	Replacement
Roots	Tool parameters , resource URIs, or server configuration
Sampling	Direct integration with LLM provider APIs
Logging	<code>stderr</code> for stdio transports; OpenTelemetry for structured observability

These are annotation-only deprecations. The methods, types, and capability flags continue to work in this release and in every specification version published within a year of it, and removing any of them will require a separate SEP under the lifecycle policy.

Full JSON Schema 2020-12 for Tools

Tool `inputSchema` and `outputSchema` are lifted to full [JSON Schema 2020-12](#) ([SEP-2106](#)). Input schemas keep the `type: "object"` root constraint but now allow composition (`oneOf` , `anyOf` , `allOf`), conditionals, and references (`$ref` , `$defs`). Output schemas are unrestricted, and `structuredContent` can now be any JSON value rather than only an object. Implementations must not auto-dereference external `$ref` URIs and should bound schema depth and validation time.

Separately, the error code for a missing resource changes from the MCP-custom `-32002` to the JSON-RPC standard `-32602 Invalid Params` ([SEP-2164](#)). If your client matches on the literal `-32002` value, update it.

How the Protocol Evolves From Here

This release contains breaking changes. We don't intend for that to be the norm.

Three governance SEPs in this release are designed so that future revisions can evolve the protocol without breaking core capabilities. The [feature lifecycle policy](#) gives every feature an *Active*, *Deprecated*, and *Removed* lifecycle with at least twelve months between deprecation and the earliest possible removal. The [Extensions framework](#) means new capabilities can ship as opt-in extensions and stabilize there before, if ever, moving into the specification. And a Standards Track SEP can no longer reach Final status until a matching scenario lands in the [conformance suite](#) ([SEP-2484](#)), which is the same suite the new [SDK tier system](#) scores official SDKs against.

The stateless rework in this release is the kind of foundational change that needed a clean break. With it landed, and with deprecation windows and extensions as the standard tools going forward, our expectation is that implementers targeting `2026-07-28` will be able to adopt future revisions without rewriting their transport or lifecycle code.

Release Timeline and Validation

The release candidate is locked as of **May 21, 2026**. The final specification will be published on **July 28, 2026**. The ten-week window is for SDK maintainers and client

implementers to validate the changes against real workloads; under the [SDK tier system](#), Tier 1 SDKs are expected to ship support within this window.

The full release candidate is in the [draft specification](#), and the [changelog](#) will list every change against 2025-11-25 .

If you find a problem, open an issue in the [specification repository](#). For implementation questions, the relevant [Working Group](#) channel in the [contributor Discord](#) is the fastest path to an answer.

Looking Ahead

This release gives MCP the foundation we expect it to grow on for a long time: a protocol that runs statelessly on commodity HTTP infrastructure, an extensions framework where capabilities like Tasks and MCP Apps can ship on their own timeline, and a lifecycle policy that lets implementers build on 2026-07-28 knowing what they ship will keep working.

Thank you to everyone who shaped these proposals through the Working Groups and a great deal of patient review. We're looking forward to making this final with the community on July 28.

[mcp](#)[spec](#)[release](#)[protocol](#)

[Documentation](#) · [Posts](#) · [Archives](#) · [Search](#) · [Tags](#) · [GitHub](#) · [LinkedIn](#) · [RSS](#)

Copyright © Model Context Protocol a Series of LF Projects, LLC.

For web site terms of use, trademark policy and other project policies please see <https://lfprojects.org>.